

Sparse File-Level Impact Plans for Repository-Level LLMs: Output Cost, Real-Commit Fidelity, and an Agentic Localization Baseline

Anonymous submission
Anonymous affiliation
Anonymous institution

Abstract—Repository-level language models must first decide which files a requested change affects, but the explicit impact plan that encodes this decision is itself an output-budget consumer: on a 144-file candidate universe an explicit per-file policy serializes roughly 140–150 decision records per task. We study *Preserve-by-Omission*, a sparse representation in which only non-default decisions are emitted and every omitted candidate deterministically decodes to “preserve.” A controlled 16K-cap encoding ablation on six curated django CMS mechanism tasks shows the sparse form uses about 90% fewer completion tokens and 96% fewer serialized records at equal operational validity, while selection fidelity is scenario-dependent rather than universally better. On ten independent, previously unseen real historical django CMS changes (sixty cells, two arms, three nested repetitions), the output-cost advantage persists (mean completion 8,446 vs. 599 tokens; total cost \$0.298 vs. \$0.062), while paired task-level F1 differences are small and cross zero. A compact system-level comparison against the LocAgent graph-guided localization framework (pinned upstream commit 4935b557326c154bad8e8dcf3747cc8d32d1f387, OpenRouter-routed Qwen3-Coder) on the same ten tasks shows comparable-to-lower file-level F1 at roughly two orders of magnitude more inference cost, with a 50% empty/non-usable localization rate (two timeouts, one context-length failure, two completed-but-empty outcomes). Sparse impact plans offer a large and reproducible output/cost reduction for repository-level impact selection; semantic fidelity and omission risk remain central open problems.

I. INTRODUCTION

REPOSITORY-level large language models (LLMs) increasingly drive software evolution tasks such as issue resolution and code repair [2], [4]. Before any edit can be written, a model must decide *which repository-relative files are affected by the requested change*. We call this decision the *impact-selection stage*. It is a prerequisite for faithful editing: an agent that never locates a changed file cannot fix it, and an agent that proposes an excessively large scope wastes output budget and risks touching unrelated code.

Modern approaches to this stage fall into two broad families. Agentic systems [1], [4], [3] navigate the repository interactively, issuing many model calls per task; retrieval-based systems [5], [6] rank candidate files by similarity

or graph proximity. Both families measure *selection effectiveness* (how often the correct files are chosen) but rarely account for the *serialization cost* of expressing the chosen scope. For an explicit impact-plan representation over a large candidate universe, this cost can dominate the interaction budget.

This paper isolates a single, separable problem: *given the same decision semantics, how much output budget does the representation of an impact plan consume, and does the cheaper representation preserve selection fidelity?* We contribute (1) Preserve-by-Omission, a sparse explicit plan in which omitted candidates deterministically decode to “preserve”; (2) a controlled 16K-cap encoding ablation on six curated django CMS mechanism tasks; (3) a real-commit replication on ten independent historical django CMS changes; and (4) a compact system-level comparison against the LocAgent graph-guided localization framework on the same ten tasks. We find the output-cost advantage is large and reproducible, while semantic fidelity is heterogeneous and omission risk (high false-negative rate) remains a central weakness.

II. RELATED WORK

SOFTWARE-repository benchmarks such as SWE-bench [2] evaluate full issue resolution end-to-end. Within that pipeline, localization is often implicit: an agent searches the repository until it finds an edit location. SWE-agent [4] explicitly instruments a file editor, while Agentless [3] shows that a lightweight locate-then-repair pipeline can be competitive without an interactive loop.

For the localization step itself, RepoCoder [5] and RepoBench [6] frame file/line completion as ranked retrieval. CodePlan [7] plans repository changes as dependency-constrained edit chains. Most relevant to our comparison, LocAgent [1] is a graph-guided agentic framework for code localization: it parses a repository into a directed heterogeneous graph, lets an LLM agent traverse the graph and BM25 index, and reports ranked files/entities. Its published file-level accuracy (~92.7% Acc@5) is reported on its own fine-tuned checkpoint and its own localization benchmark.

Our work differs in two ways. First, we measure the *representation cost* of an explicit impact policy, which the

localization literature does not isolate: we hold decision semantics constant and vary only serialization. A full policy over a 144-path universe requires roughly 140–152 explicit records; Preserve-by-Omission reduces this to a handful. Second, we compare systems on the *same* ten real held-out tasks under a shared common evaluator, and we explicitly do not reproduce LocAgent’s published fine-tuned configuration: our P5 baseline runs the pinned upstream framework with an OpenRouter-routed Qwen3-Coder model, which makes the comparison *system-level*, not an algorithm ablation.

III. METHOD: PRESERVE-BY-OMISSION

THE input to impact selection is a repository, a candidate universe U of repository-relative paths, and a natural-language change request. An *impact plan* assigns each candidate exactly one action from a fixed vocabulary; in this study the vocabulary is {PRESERVE, REGENERATE, VALIDATE, HUMAN_REVIEW}. A *full* explicit plan serializes one decision per candidate. Because most candidates are unaffected by any single change, the full serialization is dominated by hundreds of repeated “PRESERVE” rows.

Preserve-by-Omission. The sparse representation emits only non-PRESERVE decisions; every omitted candidate is reconstructed deterministically as PRESERVE. The decoding function D_s maps the emitted decision set $E_s(\pi)$ back to a complete policy: $D_s(E_s(\pi)) = \pi$ for every valid policy. This gives an exact, lossless encoding of the decision semantics and separates *representation cost* from *decision quality*: both arms carry identical action semantics and differ only in how the plan is serialized.

Controlled treatment. We reuse one common JSON-schema prompt template and a frozen candidate-ID map. The only difference between the arms is the serialization-policy block: *Full-v2* instructs explicit decisions for all candidates; *Sparse-v2* instructs explicit non-PRESERVE decisions only. A frozen representation-equivalence check proves $D_s(E_s(\pi)) = \pi$ on all valid outputs, and a prompt-control check proves the two rendered prompts are byte-identical after stripping the policy block. The model is Qwen3-Coder-480B-A35B-Instruct [8] via OpenRouter with temperature 0, structured-output JSON schema, and a 16,384-token completion cap for both arms.

IV. EXPERIMENTAL DESIGN

WE run three complementary studies, all on the django CMS 5.0.0 repository and all scored against the same kind of reference. The historical set of changed production files for a real commit is an *observed change-set proxy*: it is evidence about what was actually edited, not a perfect semantic ground truth. Selection metrics (precision, recall, F1, FNR) compare each predicted file set against this proxy. The reference is applied evaluation-only and never appears in any prompt.

Real-commit corpus (M4A). A deterministic miner over django CMS history produced 40 clean scientific changes from the newest 6,000 ancestors (~2016–2025), with a parent-only public/hidden split and a leakage barrier that rejects any case whose visible intent already names the changed paths. Ninety-two leaked candidates were excluded, exact-duplicate and related-change removal ran on the remainder, and a metadata-only split freeze (TRAIN 24 / VALIDATION 6 / HELD_OUT_TEST 10, seed frozen before any model output) fixed the ten held-out tasks used by P1 and P5.

M1 — controlled mechanism. Six curated development/mechanism scenarios (002, 004, 005, 006, 007, 008), two arms $\times$ five repetitions = 60 cells, completion cap 16,384. This is a controlled encoding ablation: the completion cap is non-binding for both arms (0 truncations), so any output difference isolates serialization, not truncation. The six scenarios are independent task units; the five repetitions are nested observations (no $n = 30$ independence claim). A separate 4,096-cap feasibility boundary (M1A) is reported only as mechanism evidence: at that cap the Full arm truncates before emitting all required decisions while Sparse completes, which is why all semantic comparisons in this paper use the non-binding 16K cap (avoiding survivor bias toward the sparse arm).

P1 — real-commit replication. Ten independent held-out real django CMS commits (HELD_OUT_TEST), two arms $\times$ three nested repetitions = 60 cells, same 16K cap, same prompts except the serialization block. Paired task-level deltas use a 10,000-resample bootstrap over the ten independent tasks, seeded and frozen; nested repetitions are pooled per task, never treated as independent examples.

P5 — system-level LocAgent baseline. The same ten held-out tasks are run once each through the LocAgent framework [1] at pinned upstream commit 4935b557326c154bad8e8dcf3747cc8d32d1f387, using OpenRouter-routed Qwen3-Coder (the framework’s upstream protocol uses temperature 1, MRR ranking, a 900 s per-attempt timeout, and one execution per task). This is a *framework baseline*, not a reproduction of the published fine-tuned LocAgent configuration, and it is evaluated with the same common evaluator against the same observed change-set proxy. P5 cost is a normalized estimate under a frozen pricing snapshot because the backend provider was OpenRouter-routed and not proven per call.

A. Validity design

Six curated scenarios are development/mechanism evidence, not held-out confirmation; only the ten P1/P5 tasks are independent held-out data. The common evaluator applies one fixed policy (exact emitted file set), and the LocAgent-native ranking metric (Acc@K) is kept separate from the common F1 columns because the two measure different things. A per-case usage ledger records model calls, tokens, latency, and estimated cost for P5; the model-call

TABLE I

M1 CONTROLLED 16K ENCODING ABLATION (6 CURATED SCENARIOS $\times$ 2 ARMS $\times$ 5 REPS = 60 CELLS; ALL VALID).

Metric	Full-v2	Sparse-v2
Valid / cells	30/30	30/30
Truncations	0	0
Mean completion tokens	8,383	809
Mean serialized records	144.0	5.9
Precision	0.452	0.721
Recall	0.775	0.883
F1	0.571	0.794
Total API cost (USD)	0.275	0.048

count is the authoritative ledger count, never the number of persisted output files.

B. Reproducibility and auditing

Every study passes six frozen pre-benchmark validation gates (dataset, prompt, pipeline smoke, dry-run, integration, metric) before any scientific call, and an independent zero-API audit after. All raw evidence is hash-pinned: P1’s 60 per-cell raw responses carry SHA-256 sidecars; the P5 per-call usage ledger and merged ranked outputs are immutable. A strengthened zero-API audit (28 checks) verifies the common P/R/F1/FNR arithmetic from raw files, the official native Acc@K semantics, the empty-outcome failure taxonomy against raw logs, the provider-route wording, and the consistent efficiency denominator. These checks exist so that the reporting errors this paper corrects (item-hit counts labelled as task accuracy, a “5/10 timeout” taxonomy, and mixed validity denominators) fail closed in future releases.

V. RESULTS

A. M1 — Controlled mechanism

At the 16K cap both arms achieve 60/60 operational validity with zero truncations, so the representation is the only systematic difference.

Table I shows the controlled effect: Sparse-v2 reduces mean completion output by about 90% (8,383→809) and serialized records by about 96% (144.0→5.9) at identical validity and no truncation. Selection fidelity improves on four of six scenarios and degrades on two; scenario 006 is the counterexample (Sparse recall 0.333 vs. Full 0.800). We therefore describe the mechanism result as a controlled *serialization-cost* effect, not a universal semantic improvement.

B. P1 — Real-commit replication

The ten held-out real commits are substantially harder and noisier than the six curated mechanism tasks. Both arms still complete 30/30 valid cells per arm with zero truncations.

As Table II shows, the output-cost advantage transfers to unseen historical changes: Sparse-v2 cuts mean completion tokens by 7,848 (bootstrap 95% CI [−8,136,

TABLE II

P1 REAL-COMMIT HELD-OUT EVALUATION (10 TASKS $\times$ 2 ARMS $\times$ 3 NESTED REPS = 60 CELLS; ALL VALID).

Metric	Full-v2	Sparse-v2
Valid / cells	30/30	30/30
Precision	0.339	0.387
Recall	0.369	0.261
F1	0.353	0.312
FNR	0.631	0.739
Mean completion tokens	8,445.8	599.0
Mean serialized records	144.0	4.07
Total API cost (USD)	0.2976	0.0623

TABLE III

P1 PER-TASK F1 (POOLED OVER THREE NESTED REPETITIONS PER TASK). FIDELITY IS HETEROGENEOUS: THE ARMS AGREE ON EASY AND HARD TASKS BUT TRADE OFF PRECISION AND RECALL ON THE MIDDLE DIFFICULTY.

Task	Full F1	Sparse F1
4307e1b8c2e2	0.500	0.400
50c3576080be	0.333	0.143
630a50361ada	0.167	0.000
66c70394c9e1	0.214	0.545
75978fb1c3ad	1.000	1.000
8d50660e7bcf	0.929	0.571
9e33db4f4660	0.118	0.400
b39799f9fc1c	0.341	0.357
ba16eb9a1d09	0.000	0.000
fdda30c271f0	0.267	0.375

−7,597], excludes zero), serialized records by 139.9 [−143.3, −137.0], and per-task cost by \$0.0235 (CI excludes zero). Task-level $\Delta F1$ (Sparse − Full) is −0.009 with 95% CI [−0.130, +0.119], which crosses zero. Both arms have high false-negative rates (0.63 and 0.74); on these real changes omission risk is a central weakness for both representations.

Table III shows the per-task F1 values that drive the inconclusive paired delta. On the two easy tasks both arms reach F1 1.0; on the two hardest tasks both arms score 0; on the middle six tasks the arms trade off precision and recall, and neither arm dominates. This is consistent with a representation that changes the output shape (and its cost) without changing the intrinsic difficulty of each task, and it is why we report a controlled cost effect rather than a semantic superiority claim.

C. P5 — LocAgent framework baseline

Table IV reports the system-level comparison on the same ten tasks. Execution/validity denominators are explicit: P1 arms ran 10 tasks $\times$ 3 nested repetitions = 30 cells each; LocAgent ran 10 tasks $\times$ 1 execution = 10 outcomes.

LocAgent produced a usable file set on 5 of 10 tasks. The other five are fail-closed empty and are *not* all timeouts: the raw logs show two 900 s timeouts, one context-length `BadRequestError`, and two completed-but-empty outcomes (the upstream flow logged “succeed” with no

TABLE IV

P5 SYSTEM-LEVEL SHARED-TASK COMPARISON ON THE SAME TEN HELD-OUT COMMITS. LOCAGENT: PINNED UPSTREAM 4935b557326c154bad8e8dcf3747cc8d32d1f387, OPENROUTER-ROUTED QWEN3-CODER, TEMPERATURE 1, MRR, 900 S TIMEOUT, ONE EXECUTION PER TASK.

System	Tasks	Runs	Non-empty	Empty	P	F1
Full-v2	10	30	30	0	0.339	0.353
Sparse-v2	10	30	30	0	0.387	0.312
LocAgent	10	10	5	5	0.435	0.333

TABLE V

TAXONOMY OF THE FIVE EMPTY LOCAGENT OUTCOMES, CLASSIFIED FROM THE RAW EXECUTION LOGS. ALL FIVE ARE PERSISTED FAIL-CLOSED, BUT ONLY TWO ARE TIMEOUTS.

Task	Class	Log evidence
4307e1b8c2e2	900 s timeout	“exceeded timeout”
66c70394c9e1	context-length BadRequest	“maximum context length”
9e33db4f4660	completed-but-empty	“succeed”, no files
b39799f9fc1c	completed-but-empty	“succeed”, no files
fdada30c271f0	900 s timeout	“exceeded timeout”

parseable file set) — a 50% empty/non-usable localization rate, not a 50% timeout rate.

Pooled micro F1 for LocAgent (0.333) sits between Sparse (0.312) and Full (0.353); paired task-level $\Delta F1$ bootstrap CIs cross zero for both comparisons (LocAgent – Full: $-0.068 [-0.250, +0.170]$; LocAgent – Sparse: $-0.061 [-0.306, +0.241]$). LocAgent’s native ranking on the tasks that complete is Acc@1 4/10, Acc@3 4/10, Acc@5 2/10 under the official LocAgent metric; these are reported separately and are not comparable to our F1. The failure taxonomy in Table V is reported so that the robustness profile is not conflated with a single timeout mechanism.

The efficiency contrast is large and is stated with one consistent denominator (mean per execution/task):

As Table VI shows, LocAgent uses about $246\times$ the mean tokens and $100\times$ the mean cost of Full-v2 (and about $594\times / 478\times$ of Sparse-v2), totaling 402 model calls and roughly \$9.93 of normalized estimated cost (32.8M tokens) versus \$0.30 (Full) and \$0.06 (Sparse). On these ten tasks the accuracy–inference-cost frontier does not favor the graph-agent baseline; the cost advantage of Preserve-by-Omission is not offset by superior localization accuracy.

VI. DISCUSSION

THE evidence supports three claims. First, the representation of an impact plan is a first-class output-budget cost: on a 144-path universe an explicit full policy costs roughly an order of magnitude more output than a sparse one, and this holds in a controlled ablation with a non-binding cap. Second, this cost advantage transfers to independent real historical changes: Sparse-v2 reduces completion tokens, serialized records, and cost with bootstrap CIs excluding zero, at small and statistically inconclusive F1 deltas. Third, at the system level, an

TABLE VI

MEAN PER-EXECUTION/TASK INFERENCE EFFICIENCY ON THE SAME TEN TASKS. ONE CONSISTENT DENOMINATOR: P1 MEANS ARE PER CELL ($n = 30$), LOCAGENT MEANS ARE PER TASK EXECUTION ($n = 10$). P5 COST IS A NORMALIZED ESTIMATE UNDER A FROZEN PRICING SNAPSHOT.

System	Mean total tokens	Mean cost (USD)	Calls
Full-v2	13,362	0.00992	30
Sparse-v2	5,530	0.00208	30
LocAgent	3,283,177	0.99288	402

interactive graph-guided agent does not deliver a clear accuracy advantage on these ten tasks at its much higher inference cost.

These findings have a design implication for repository-level editing. The high false-negative rate of both our arms (0.63–0.74) means the cheap sparse plan should not be trusted as final scope. A cost-effective architecture would use a sparse first-pass plan, flag omission-risk candidates, and escalate only those for bounded verification — rather than running an always-on high-cost agent on every task. LocAgent’s graph traversal suggests dependency proximity as a useful soft signal for that escalation, but no graph-assisted inference was executed in this study, so we make no causal claim about graph benefit. The P5 result also illustrates a system-level trade-off that is independent of the sparse representation: an interactive localization agent can spend two orders of magnitude more inference budget without a corresponding accuracy gain, which strengthens the case for budget-aware selection strategies.

VII. THREATS TO VALIDITY

WE consolidate the main threats. (1) The observed change-set proxy is not perfect semantic ground truth; the historical diff may under- or over-state the files a model “should” flag. (2) The ten held-out tasks come from a single repository (django CMS); external generalization to other repositories is not measured. (3) M1 uses six curated scenarios as independent units; five repetitions are nested, so $n = 6$ is the inferential unit for the controlled study. (4) Semantic effects are heterogeneous, with scenario 006 degrading under the sparse arm; no universal superiority claim is made. (5) The P1 $\Delta F1$ CIs cross zero, so we do not claim a detected difference — and crossing zero is not evidence of equivalence either. (6) The P5 comparison is system-level, not an algorithm ablation: protocols, temperature, interaction budget, and model serving differ; LocAgent’s backend provider was OpenRouter-routed and not proven per call, and its cost is a normalized estimate under a frozen pricing snapshot. (7) The 4,096-cap feasibility boundary is not used for semantic comparison because it censors the Full arm (survivor bias); all semantic comparisons use the non-binding 16K cap. (8) High FNR is a weakness, not a success: both representations omit a substantial fraction of observed proxy files.

VIII. CONCLUSION

WE introduced Preserve-by-Omission, a sparse explicit impact plan that deterministically reconstructs a complete policy from only its non-default decisions. In a controlled 16K-cap encoding ablation it reduces completion output by about 90% and serialized records by about 96% at equal operational validity; on ten independent real historical django CMS commits the output/cost advantage persists while semantic fidelity remains heterogeneous and paired F1 differences are statistically inconclusive. A compact system-level comparison shows the LocAgent graph-guided framework does not outperform the cheap sparse planner on accuracy while costing roughly two orders of magnitude more inference. The central open problem is omission risk: high false-negative rates motivate selective, bounded verification of low-cost impact plans rather than always-on agentic search. All experiments, scores, and audits are reproducible from frozen raw evidence in the public repository.

REFERENCES

- [1] Z. Chen et al., “LocAgent: Graph-guided LLM agents for code localization,” in *Proc. ACL (Long Papers)*, 2025.
- [2] C. E. Jimenez et al., “SWE-bench: Can language models resolve real-world GitHub issues?” in *Proc. ICLR*, 2024.
- [3] C. S. Xia et al., “Agentless: Demystifying LLM-based software engineering agents,” *arXiv:2407.01489*, 2024.
- [4] J. Yang et al., “SWE-agent: Agent-computer interfaces enable automated software engineering,” in *Proc. NeurIPS*, 2024.
- [5] F. Zhang et al., “RepoCoder: Repository-level code completion through iterative retrieval and generation,” in *Proc. EMNLP*, 2023.
- [6] R. Tian et al., “RepoBench: Benchmarking repository-level code auto-completion systems,” in *Proc. ICLR*, 2024.
- [7] R. Bairi et al., “CodePlan: Repository-level coding using LLMs and planning,” in *Proc. NeurIPS*, 2023.
- [8] Qwen Team, “Qwen3 technical report,” *arXiv:2505.09388*, 2025.